

Do Texto Bruto ao Conhecimento Compilado: Arco Evolutivo do CKF como Tecnologia de Compilação de Conhecimento

Uma síntese narrativa, técnica e científica dos doze estudos da série CKF

Autor: Paulo Tomazinho, PhD

Organização: CKF Research

Data: Maio de 2026

Resumo

Este artigo sintetiza o arco científico e tecnológico do Compiled Knowledge Format (CKF) a partir de doze estudos sequenciais. A série começou com a nomeação de um modo de falha em sistemas RAG — a **composition hallucination**, isto é, o erro em que os fragmentos corretos estão presentes, mas o modelo falha ao compor suas relações — e evoluiu para a construção, avaliação, regressão, correção, reprodutibilidade e comparação entre modelos do **CKF Compiler**. O percurso mostra que CKF não deve ser entendido apenas como um formato de serialização, mas como uma tecnologia de **compilação de conhecimento**: um pipeline que transforma documentos humanos em uma representação intermediária estruturada, rastreável, auditável e consumível por agentes. Ao longo da série, a pergunta central deixou de ser “um CKF pode ser gerado?” e passou por estágios sucessivos: “o conhecimento é preservado?”, “as unidades são alocadas nas seções corretas?”, “a sanitização melhora ou destrói o pacote?”, “a cadeia de ingestão preserva a estrutura do documento?”, “a geração é reprodutível?”, “a preservação é estável?”, “o resultado depende do modelo?”, e finalmente “qual configuração é adequada para extração científica de alta fidelidade?”. A série mostra que a qualidade CKF é multidimensional: preservação semântica, densidade estrutural, section allocation, retrieval readiness, rastreabilidade, fidelidade linguística, observabilidade, ingestão de PDF, granularidade de chunking, escolha de modelo e configuração de execução podem melhorar ou regredir independentemente. A conclusão integradora é que CKF evoluiu de uma hipótese sobre retrieval estruturado para uma metodologia de compilação preservacionista de conhecimento. O próximo estágio não é simplesmente “gerar mais JSON”, mas criar garantias de compilação: modos de extração, manifests de execução, testes de regressão semântica, quality gates por finalidade e avaliação downstream em tarefas reais de RAG e agentes.

Palavras-chave: CKF; Compiled Knowledge Format; compilação de conhecimento; composition hallucination; preservação semântica; RAG; GraphRAG; MCP; rastreabilidade; reprodutibilidade; LLMs; representação intermediária; conhecimento estruturado; agentes de IA.

1. Introdução

A história científica do CKF começa com um incômodo prático: sistemas RAG frequentemente falham mesmo quando a informação correta está presente. A hipótese simplificada de muitos pipelines era: se

o retriever trouxe o chunk certo, o modelo responderá corretamente. Mas, em documentos normativos, técnicos, jurídicos, educacionais ou operacionais, o conhecimento raramente está contido em uma única frase. Ele depende de relações: regra geral, exceção, exceção da exceção, condição de aplicabilidade, precedência, sequência procedural, restrição, obrigação, escopo e conflito.

O CKF nasce dessa lacuna. Seu ponto de partida não é a criação de mais um formato de arquivo, mas a percepção de que documentos humanos e agentes de IA usam representações diferentes. PDFs, páginas Markdown, manuais e artigos científicos codificam conhecimento por meio de prosa, títulos, contraste, layout, exemplos e relações implícitas. Agentes operam sobre spans, chunks, schemas, context windows, tool calls, provenance e estruturas recuperáveis.

A pergunta inicial era simples:

Como evitar que um agente tenha acesso às frases corretas, mas falhe ao compor o conhecimento?

A resposta proposta pelo CKF evoluiu ao longo da série:

mover parte da inferência estrutural do tempo de inferência para o tempo de compilação.

Assim, CKF passou a ser entendido como uma **representação intermediária compilada para conhecimento**. Documentos humanos são a fonte. Pacotes CKF são o IR. Agentes, sistemas RAG, bancos vetoriais, graph stores e servidores MCP são os runtimes.

Este artigo é a síntese dos doze estudos que construíram essa tese.

2. A tese central: CKF como compilador, não como serializador

A série mostra uma mudança conceitual fundamental.

No início, era natural perguntar:

“O CKF é um JSON melhor?”

Ao final da série, essa pergunta se mostrou insuficiente. CKF não é apenas JSON, YAML ou Markdown. Esses são formatos de serialização. O CKF é melhor compreendido como o resultado de um **compilador de conhecimento**.

A comparação mais adequada é com compiladores de software:

Software	Conhecimento CKF
Código-fonte humano	Documento humano
Parser/compiler frontend	PDF/text extraction + chunker

Software	Conhecimento CKF
Intermediate Representation	Pacote CKF
Optimization passes	reduce, promotion, sanitation, dedup, traceability rebuild
Runtime/backend	RAG, agente, MCP, vector DB, graph store

Essa analogia amadureceu ao longo da série. Ela não surgiu como metáfora decorativa. Ela se tornou explicativa: um compilador precisa preservar semântica, estruturar corretamente, manter rastreabilidade, evitar regressões, aplicar passes na ordem correta, testar invariantes e registrar manifests de execução.

O aprendizado central é:

CKF deve ser avaliado como compilação, não como conversão de formato.

3. Artigo 1 — Composition hallucination: nomear o problema

Pergunta

O primeiro estudo perguntou:

Que tipo de erro ocorre quando RAG recupera os trechos corretos, mas o modelo ainda responde errado porque não compõe as relações entre eles?

O que fizemos

O estudo nomeou e descreveu a **composition hallucination**. O exemplo central é simples: uma política diz que gerentes podem aprovar despesas até certo valor, mas outra cláusula diz que despesas internacionais exigem aprovação do CFO. Se o modelo recupera ambas as cláusulas e ainda responde que o gerente pode aprovar a despesa internacional, a falha não é paramétrica nem de retrieval. A informação estava presente. O erro está na composição.

O que descobrimos

A série começou distinguindo quatro classes de falha:

1. falha paramétrica: o modelo inventa;
2. falha de retrieval: o trecho correto não foi recuperado;
3. falha contextual: o trecho foi recuperado, mas ignorado;
4. falha composicional: os trechos corretos estão presentes, mas a relação entre eles não é integrada.

A quarta classe era a lacuna conceitual que motivou CKF.

O que aprendemos

Aumentar top-k, aumentar janela de contexto ou recuperar mais chunks não resolve necessariamente a falha. Em conhecimento normativo e operacional, o problema é representacional: relações precisam ser explicitadas antes ou durante a recuperação.

Nova pergunta criada

Se composition hallucination é uma falha de composição relacional, uma representação estruturada do conhecimento pode reduzir esse erro?

4. Artigo 2 — CKF versus texto bruto em RAG: o piloto exploratório

Pergunta

O segundo estudo perguntou:

CKF melhora respostas em RAG quando comparado a texto bruto?

O que fizemos

Foi conduzido um piloto com dez perguntas em português sobre três representações do mesmo corpus: TXT bruto, PDF-derived raw e CKF. O pipeline usou o mesmo agente, mesmo judge, Gemini 3 Flash Preview, temperatura 0.2 e orçamento de contexto generoso.

O que descobrimos

Todos os formatos performaram perto do teto: médias de aproximadamente 0,974 para TXT, 0,968 para PDF e 0,964 para CKF. Não houve diferença estatisticamente significativa. CKF, porém, produziu saídas mais curtas e menor latência descritiva.

O que aprendemos

O piloto não falsificou nem confirmou a hipótese CKF. Ele revelou um **ceiling effect**: em um corpus simples, com contexto abundante e perguntas tratáveis, todas as condições funcionam bem.

Nova pergunta criada

Como desenhar um Study 2 confirmatório que realmente coloque RAG sob retrieval pressure?

A resposta metodológica foi: reduzir contexto, reduzir top-k, usar perguntas multi-hop, exigir composição, usar judge independente e tornar a hipótese falsificável.

5. Artigo 3 — Primeira análise de preservação: formato estruturado não basta

Pergunta

O terceiro estudo perguntou:

Quando um manifesto narrativo é convertido em CKF JSON, Markdown e YAML, quanto conhecimento substantivo é preservado?

O que fizemos

Foi criada uma rubrica de 20 unidades substantivas. Essa rubrica não media frases literais nem perguntas. Media proposições centrais que precisariam sobreviver para que o documento compilado mantivesse seu significado.

O que descobrimos

Os artefatos estruturados preservavam apenas cerca de 17,5% da rubrica. Eles eram mais legíveis por máquina, mas semanticamente rasos. Muitas ideias centrais desapareciam: composition failures, relações com RAG/GraphRAG/MCP, provenance como infraestrutura de auditoria, benchmarking confirmatório e falsificabilidade.

O que aprendemos

A descoberta foi decisiva:

schema válido não é preservação de conhecimento.

JSON válido, YAML compacto e Markdown legível não garantem preservação semântica.

Nova pergunta criada

Que arquitetura de compiler é necessária para preservar conhecimento, e não apenas gerar estrutura?

6. Artigo 4 — CKF v1.0: o salto de preservação

Pergunta

O quarto estudo perguntou:

Quando o CKF Compiler v1.0 é usado, quanto conhecimento substantivo é preservado e quais mecanismos explicam isso?

O que fizemos

Analizamos o PDF canônico *What CKF is not (and what it actually is)* e artefatos CKF em JSON, Markdown e YAML. Também auditamos arquivos TypeScript do compiler: extraction schema, semantic chunker, reducer, quality gates, provider adapters e compile pipeline.

O que descobrimos

A preservação saltou de cerca de 10%/17,5% nos outputs rasos para aproximadamente **97,5%**. O v1.0 extrai 15 entidades, 14 conceitos, 56 atomic units, 23 retrieval chunks e 112 source-traceability links.

O que aprendemos

A melhora não veio da serialização. Veio da arquitetura:

- chunking semântico;
- stable span IDs;
- structured tool calling;
- source excerpts;
- confidence scoring;
- canonical reduction;
- quality gates.

Mas uma nova limitação apareceu: o conhecimento era preservado, mas ficava concentrado em `atomic_units` e `retrieval_chunks`.

Nova pergunta criada

Preservar conhecimento é suficiente, ou precisamos alocar o conhecimento nas seções corretas do schema?

7. Artigo 5 — CKF v1.01: section allocation e promotion

Pergunta

O quinto estudo perguntou:

Um compiler pode preservar 100% do conteúdo e alocar claims em seções normativas e operacionais mais ricas?

O que fizemos

Avaliamos o compiler v1.01, ainda compatível com `ckf-1.0`, mas com uma nova passagem de **promotion**. A ideia era duplicar unidades atômicas e retrieval chunks elegíveis em seções como `principles`, `decision_rules`, `procedures`, `anti_patterns`, `exceptions` e `qa_pairs`.

O que descobrimos

O v1.01 atingiu **100% de preservação semântica**. O total contado subiu para 388 elementos. As seções ricas começaram a ser populadas: 25 principles, 9 decision rules, 4 procedures, 8 anti-patterns, 5 exceptions, 27 QA pairs, 21 retrieval chunks, 64 atomic units e 194 traceability items.

O que aprendemos

O problema da v1.0 não era falta de extração, mas **schema allocation**. Uma mesma ideia pode sobreviver no pacote, mas estar em uma seção pouco útil para agentes.

O insight arquitetural foi:

promotion deve duplicar, não migrar.

Dessa forma, preserva-se a retrieval surface ao mesmo tempo em que se melhora a representação normativa e operacional.

Nova pergunta criada

Como medir qualidade de alocação e evitar artifacts gerados por promotion?

8. Entre v1.01 e v1.02 — qualidade, metadata e primeiras regras condicionais

Embora a série numerada enfatize alguns marcos, a v1.02 foi conceitualmente importante.

Pergunta

Como tornar a qualidade CKF observável e calibrada?

O que fizemos

Introduzimos ou consolidamos métricas como:

- `human_readability`;
- `ai_utility_score`;
- `section_allocation_quality`;
- `language_fidelity`;
- `promotion_acceptance_rate`.

Também surgiram melhorias em playbooks e if-then rules.

O que descobrimos

A preservação permaneceu em 100%, mas language drift e artifacts truncados ainda apareciam.

O que aprendemos

Filtros aplicados apenas às novas promoções não bastam. O pacote inteiro precisa de sanitation.

Nova pergunta criada

Como limpar globalmente o pacote sem destruir conteúdo válido?

9. Artigo 6 — CKF v1.03: sanitation global e a regressão

Pergunta

O sexto estudo perguntou:

A sanitização global resolve language drift e artifacts truncados?

O que fizemos

O v1.03 introduziu:

- global package sanitation;
- language lock no prompt;
- filtros de idioma;
- truncation filtering;
- cleanup pós-reduce e pós-promotion.

O que descobrimos

A v1.03 resolveu problemas visíveis: drift linguístico e artifacts truncados desapareceram. Mas introduziu uma regressão grave: removeu todos os `retrieval_chunks`, `procedures`, `if_then_rules` e `playbooks`.

O que aprendemos

Sanitização global é necessária, mas perigosa. O erro foi aplicar regras de completude proposicional a campos que não são proposições, como títulos, labels, nomes e tags.

O aprendizado foi uma das teses técnicas mais importantes da série:

sanitation precisa ser field-aware.

Nova pergunta criada

Como construir um sanitizer que remova ruído sem destruir a retrieval surface?

10. Artigo 7 — CKF v1.03.1: equilíbrio maduro

Pergunta

O sétimo estudo perguntou:

A v1.03.1 corrige a regressão de over-sanitization mantendo os ganhos de limpeza?

O que fizemos

A v1.03.1 introduziu:

- sanitizer sensível ao papel do campo;
- distinção entre `label`, `title`, `name`, `tag`, `substantive` e `source_excerpt`;
- preservação de retrieval chunks quando o corpo é válido;
- isenção de source excerpts em filtros de idioma;
- `ensureIds()`;
- `rebuildSourceTraceability()` após sanitation.

O que descobrimos

A v1.03.1 atingiu novamente **100% de preservação semântica**, com 511 elementos contados, 18 retrieval chunks, 66 atomic units, 4 procedures, 3 playbooks, 36 QA pairs e 261 source traceability entries.

O que aprendemos

A v1.03.1 foi o primeiro ponto de equilíbrio maduro:

- preservação plena;
- section allocation forte;
- retrieval readiness recuperada;
- language drift corrigido;
- truncation filtering mantido;
- rastreabilidade reconstruída.

Nova pergunta criada

Se o compiler lógico está correto, por que o ambiente público pode gerar pacotes rasos?

11. Artigo 8 — Quando o compiler está correto, mas o pacote sai raso

Pergunta

O oitavo estudo perguntou:

Por que o mesmo PDF canônico gerou output muito mais raso no Lab/Compiler público, mesmo usando o compiler v1.1?

O que fizemos

Investigamos a cadeia operacional: rota `/lab/new`, `/compiler`, PDF extraction, chunking, modelo, labels de UI, pipeline trace e contadores por etapa.

O que descobrimos

O problema não era necessariamente o compiler antigo. O pipeline v1.1 estava correto. O problema estava antes ou ao redor dele:

- extração de PDF removia headings e parágrafos;
- chunk size grande demais reduzia granularidade;
- modelo Flash era otimizado para velocidade, não profundidade;
- UI confundia `protocol_version: ckf-1.0` com `compiler_deployment_version: v1.1`;
- faltava observabilidade de pipeline.

O que aprendemos

A tese amadureceu:

CKF quality is end-to-end quality.

A qualidade do pacote depende de toda a cadeia:

```
source document
→ structure-preserving extraction
→ semantic chunking
→ model extraction
→ reduce
→ promotion
→ sanitation
→ traceability rebuild
→ quality gates
→ UI observability
```

Nova pergunta criada

Depois de corrigir ingestão e observabilidade, a geração é estável e reprodutível?

12. Artigo 9 — Reprodutibilidade macroestrutural com Gemini 3 Flash

Pergunta

O nono estudo perguntou:

Se o mesmo PDF for compilado três vezes com o mesmo modelo e mesma configuração, o pacote CKF é estável?

O que fizemos

Executamos três compilações independentes com CKF Compiler v1.1 e Gemini 3 Flash Preview.

O que descobrimos

As três execuções foram macroestruturalmente estáveis:

- total contado variou de 464 a 472;
- source traceability variou de 237 a 243;
- atomic units variaram de 54 a 58;
- retrieval chunks de 14 a 17;
- QA pairs de 33 a 36.

A variação total foi inferior a 2%. Porém, `procedures` e `playbooks` variaram mais, e `if_then_rules` permaneceu zerado.

O que aprendemos

Reprodutibilidade CKF precisa ser dividida em níveis:

1. **reprodutibilidade estrutural;**
2. **reprodutibilidade operacional;**
3. **reprodutibilidade determinística.**

O v1.1 com Gemini Flash é estruturalmente reprodutível, mas não item-determinístico.

Nova pergunta criada

A preservação semântica também é estável entre execuções?

13. Artigo 10 — Preservação semântica em três execuções Flash

Pergunta

O décimo estudo perguntou:

As três execuções independentes com Gemini 3 Flash preservam semanticamente o documento canônico?

O que fizemos

Aplicamos novamente a rubrica de 20 unidades às três execuções A, B e C.

O que descobrimos

Os scores foram:

Execução	Score	Preservação
A	19/20	95%
B	19,5/20	97,5%
C	19,5/20	97,5%
Média	19,33/20	96,7%

O núcleo conceitual foi preservado. As perdas ocorreram em detalhes finos: `Study 2`, `pre-registered` e, em uma execução, `retrieval pressure`.

O que aprendemos

Gemini Flash é semanticamente robusto para preview e estudos estruturais, mas não maximal para publicação científica.

Nova pergunta criada

A diferença vem do modelo? Modelos mais fortes preservam melhor detalhes científicos?

14. Artigo 11 — Sensibilidade a modelo

Pergunta

O décimo primeiro estudo perguntou:

Entre modelos funcionais, como a escolha do modelo afeta profundidade, rastreabilidade e preservação semântica?

O que fizemos

Comparamos GPT-5, GPT-5 mini e Gemini 3 Flash Preview no mesmo PDF canônico. Excluímos uma execução anômala com Gemini 2.5 Flash por sinais de falha operacional.

O que descobrimos

Modelo	Preservação	Total estrutural	Traceability
GPT-5	100%	846	433
GPT-5 mini	95%	478	245
Gemini 3 Flash	95%	392	203

GPT-5 preservou integralmente `Study 2`, `pre-registered`, `retrieval pressure`, `LLVM IR` e `intermediate representation`. GPT-5 mini e Gemini 3 Flash preservaram o núcleo, mas perderam detalhes científicos finos.

O que aprendemos

A versão do compiler não determina sozinha a preservação. A qualidade depende de:

```
compiler version
+ model
+ provider
+ extraction mode
+ chunking
+ prompt compliance
+ sanitation
+ language lock
```

Nova pergunta criada

Entre modelos fortes, GPT-5 continua superior ou Gemini 2.5 Pro alcança preservação research-grade?

15. Artigo 12 — GPT-5 versus Gemini 2.5 Pro

Pergunta

O décimo segundo estudo perguntou:

GPT-5 e Gemini 2.5 Pro, como modelos fortes, preservam a rubrica semântica e os detalhes científicos do programa CKF?

O que fizemos

Comparamos duas compilações do mesmo PDF canônico com CKF Compiler v1.1: uma com GPT-5 e outra com Gemini 2.5 Pro.

O que descobrimos

Modelo	Preservação	Total estrutural	Source traceability
GPT-5	20/20 = 100%	846	433
Gemini 2.5 Pro	19,5/20 = 97,5%	432	221

Ambos preservaram o núcleo CKF. A diferença crítica: GPT-5 preservou explicitamente `Study 2` e `pre-registered`; Gemini 2.5 Pro preservou LLVM IR e intermediate representation, mas não preservou claramente a referência ao Study 2 pré-registrado.

O que aprendemos

Gemini 2.5 Pro é forte e adequado para pacotes finais equilibrados. GPT-5 é superior para **research-grade extraction**, especialmente quando o documento contém programa científico, estudos nomeados, pre-registros e hipóteses empíricas.

Nova pergunta criada

Como tornar preservação research-grade menos dependente do modelo?

A resposta proposta:

- guardrails para named studies;
- metadata repair pass;
- teste de regressão para Study 2/pre-registered;
- modos `compact`, `standard`, `deep`, `research-grade` ;
- run manifest completo.

16. O arco evolutivo do CKF em uma tabela

Etapa	Pergunta dominante	Resultado	Nova pergunta
1	Que erro RAG não explica?	Composition hallucination	Representação estruturada ajuda?
2	CKF melhora RAG?	Piloto com ceiling effect	Como testar sob retrieval pressure?

Etapa	Pergunta dominante	Resultado	Nova pergunta
3	Estrutura preserva conhecimento?	Não: ~17,5%	Como criar compiler preservation-aware?
4	v1.0 preserva?	97,5%	Como melhorar section allocation?
5	v1.01 aloca melhor?	100%, promotion	Como medir/limpar qualidade?
6	v1.03 sanitiza?	Sim, mas destrói retrieval surface	Como sanitizar por campo?
7	v1.03.1 corrige?	100%, 511 elementos	Por que Lab público sai raso?
8	Pipeline end-to-end importa?	Sim: PDF/chunk/model/UI	A geração é reproduzível?
9	A estrutura é estável?	Sim, macroestruturalmente	A semântica é estável?
10	A semântica é estável?	Sim, 95–97,5%	Modelos fortes melhoram?
11	Modelo importa?	Sim: GPT-5 chega a 100%	Gemini Pro alcança GPT-5?
12	GPT-5 vs Gemini 2.5 Pro	Ambos fortes; GPT-5 é research-grade	Como reduzir dependência de modelo?

17. O que a série provou

A série não prova apenas que CKF “funciona”. Ela prova algo mais específico e tecnicamente mais importante:

17.1 Estrutura sintática não basta

Os primeiros outputs CKF mostraram que JSON, YAML e Markdown podem ser válidos e ainda assim semanticamente pobres.

17.2 Preservação semântica é mensurável

A rubrica de 20 unidades transformou preservação em objeto avaliável.

17.3 A arquitetura do compiler importa

Chunking semântico, spans estáveis, source excerpts, confidence, reducer e quality gates explicaram o salto da v1.0.

17.4 Section allocation é uma dimensão própria

Preservar uma ideia em `atomic_units` não equivale a alocá-la como princípio, regra, exceção, procedimento ou playbook.

17.5 Sanitização pode melhorar ou destruir

A v1.03 mostrou que limpeza global sem sensibilidade de campo pode matar a retrieval surface.

17.6 Rastreabilidade precisa ser reconstruída após limpeza

A v1.03.1 mostrou que source traceability deve ser uma invariável pós-sanitização, não uma etapa anterior e imutável.

17.7 Qualidade CKF é end-to-end

PDF extraction, chunking, modelo, compiler passes e UI observability são todas partes do sistema.

17.8 Reprodutibilidade tem níveis

CKF pode ser estável estruturalmente sem ser determinístico item-a-item.

17.9 Modelo importa

O mesmo compiler v1.1 pode preservar 95%, 97,5% ou 100% dependendo do modelo e da configuração.

17.10 Research-grade extraction é uma finalidade específica

Publicação científica exige preservar estudos nomeados, hipóteses, benchmarks, pre-registros e condições experimentais.

18. O conceito maduro: CKF como compilação preservacionista

Ao final da série, CKF pode ser definido com mais precisão:

CKF é uma tecnologia de compilação preservacionista de conhecimento que transforma documentos humanos em representações intermediárias tipadas, rastreáveis e operacionalmente úteis para agentes, retrieval systems e protocolos de contexto.

Essa definição tem quatro partes.

18.1 Compilação

CKF não é apenas conversão de arquivo. Há passes, invariantes, regressões, quality gates e manifests.

18.2 Preservação

A unidade de avaliação não é o byte nem a seção, mas a proposição substantiva do documento-fonte.

18.3 Representação intermediária

CKF fica entre documentos humanos e runtimes agentivos.

18.4 Operacionalidade

CKF não preserva apenas o que o documento diz, mas como o conhecimento deve operar: regra, exceção, procedimento, anti-pattern, QA, retrieval unit, provenance.

19. A sequência técnica do compiler como hipótese científica

A série sugere que um compiler CKF maduro precisa seguir uma ordem de passes:

```
source document
→ structure-preserving extraction
→ semantic chunking
→ per-chunk structured extraction
→ reduce / canonical merge
→ promotion
→ field-aware sanitation
→ ensureIds
→ rebuildSourceTraceability
→ quality gates
→ serialization
→ reproducibility manifest
```

Cada etapa tem uma pergunta científica:

Passo	Pergunta
extraction	A estrutura do documento sobreviveu?
chunking	A unidade semântica está bem delimitada?
extraction	O modelo extraiu claims e provenance?
reduce	As partials foram integradas sem perda?

Passo	Pergunta
promotion	Claims foram alocados em seções úteis?
sanitation	Ruído foi removido sem destruir conteúdo?
traceability	Todos os itens apontam para fonte válida?
quality	O pacote é útil para humanos e agentes?
manifest	A execução é reprodutível?

20. As grandes perguntas que a série criou

A série respondeu muitas perguntas, mas abriu outras mais maduras.

20.1 Study 2: CKF melhora RAG sob pressão?

A pergunta fundacional ainda precisa de confirmação downstream:

CKF reduz composition hallucination e melhora faithfulness/precision sob retrieval pressure?

O piloto mostrou ceiling effect. O Study 2 deve ser restritivo, multi-hop e falsificável.

20.2 Como medir utilidade agentiva?

Preservação do pacote é necessária, mas não suficiente. Precisamos medir:

- resposta correta;
- rastreabilidade da resposta;
- uso de exceções;
- aplicação de procedimentos;
- resistência a conflitos normativos;
- token efficiency;
- latency;
- auditability.

20.3 Como reduzir sensibilidade ao modelo?

GPT-5 alcança 100%. Gemini 2.5 Pro chega a 97,5%. Flash fica em 95–97,5%. A pergunta agora é:

Como fazer o compiler proteger detalhes críticos mesmo com modelos menores?

20.4 Como definir modos de compilação?

A série sugere perfis:

Modo	Uso
<code>preview</code>	inspeção rápida
<code>standard</code>	pacote geral equilibrado
<code>deep</code>	produção de alta cobertura
<code>research-grade</code>	paper, benchmark, reprodutibilidade

20.5 Como criar garantias de pacote?

A próxima fase deve ter invariantes:

- semantic preservation threshold;
- retrieval surface minimum;
- source traceability no orphan IDs;
- language fidelity;
- metadata completeness;
- named study preservation;
- run manifest completeness.

21. Limitações da série

A série é forte como programa longitudinal de desenvolvimento, mas tem limitações claras.

1. Muitos estudos usam um documento canônico único.
2. A rubrica de 20 unidades depende de julgamento humano.
3. Várias comparações usam uma execução por modelo.
4. Nem todos os estudos medem desempenho downstream.
5. Os resultados dependem de provider, modelo, configuração, prompt, chunking e extração de PDF.
6. A avaliação de qualidade ainda combina métricas objetivas e interpretação semântica.

Essas limitações não enfraquecem o projeto; elas definem sua próxima agenda.

22. O artigo mais importante: a tese final da série

A maior contribuição da série não é um número isolado como 100% de preservação, 846 elementos ou 433 traceability entries.

A maior contribuição é a construção de uma nova unidade de análise:

o compilador de conhecimento.

A série mostra que, para agentes de IA, o documento não deve ser apenas lido, resumido ou chunkado. Ele pode ser compilado.

E, se pode ser compilado, então pode ser avaliado como compilação:

- preserva semântica?
- mantém provenance?
- aloca corretamente?
- expõe estrutura operacional?
- resiste a sanitization destrutiva?
- mantém retrieval readiness?
- é reprodutível?
- é sensível ao modelo?
- tem manifest?
- melhora downstream?

Essa é a tese integradora:

CKF transforma conhecimento documental em representação intermediária operacional, e sua qualidade deve ser medida por uma ciência de compiladores de conhecimento.

23. Conclusão

O arco dos doze estudos mostra uma evolução rara: o projeto não seguiu uma linha reta de sucesso, mas uma sequência científica de hipótese, falha, método, melhoria, regressão, correção, instrumentação e comparação.

Começamos nomeando uma falha: **composition hallucination**.

Testamos CKF contra texto bruto e descobrimos um ceiling effect.

Criamos uma rubrica de preservação e descobrimos que estrutura válida não bastava.

Construímos um compiler v1.0 e atingimos 97,5% de preservação.

Criamos promotion na v1.01 e chegamos a 100%.

Instrumentamos qualidade na v1.02.

Sanitizamos globalmente na v1.03 e descobrimos over-sanitization.

Corrigimos com field-aware sanitizer na v1.03.1.

Percebemos que o Lab público dependia de PDF extraction, chunking, modelo e UI observability.

Medimos reprodutibilidade macroestrutural.

Medimos preservação semântica entre execuções.

Comparamos modelos e demonstramos sensibilidade.

Finalmente, mostramos que GPT-5 atinge research-grade preservation, enquanto Gemini 2.5 Pro chega perto, mas ainda perde detalhes científicos finos.

A conclusão final é:

CKF amadureceu de uma proposta de formato para uma tecnologia de compilação de conhecimento.

E a nova pergunta científica não é mais “CKF preserva conhecimento?” — a série já mostrou que sim, sob condições adequadas.

A nova pergunta é:

Em que condições CKF melhora o desempenho real de agentes e sistemas RAG sob pressão de recuperação, mantendo rastreabilidade, auditabilidade e eficiência?

Essa é a próxima fronteira do projeto.

24. Declaração de disponibilidade

Os materiais da série incluem os doze artigos numerados, os PDFs canônicos, pacotes CKF JSON/Markdown/YAML, rubricas de preservação, scripts de contagem estrutural, análises de rastreabilidade, estudos de reprodutibilidade, comparações por modelo e recomendações de evolução do compiler.

25. Declaração de conflito de interesse

O autor é afiliado à CKF Research, organização responsável pelo desenvolvimento do Compiled Knowledge Format e do CKF Compiler. A série documenta tanto resultados positivos quanto falhas, regressões e limitações observadas durante o desenvolvimento.

26. Contribuição do autor

Paulo Tomazinho concebeu o CKF, formulou o conceito de composition hallucination, conduziu os estudos de preservação, comparou versões do compiler, avaliou reprodutibilidade e sensibilidade a modelos, e consolidou este artigo de síntese.